

52. Devant, derrière : à toi de choisir, avec A

Le héros se déplace avec les flèches ; A le fait passer devant ou derrière le mur. On voit la différence en direct.

1. Ce qu'il faut comprendre

Cette fois, c'est toi qui décides. Les quatre flèches déplacent le héros. Le bouton **A** bascule entre « devant » et « derrière ». Va sur le mur, appuie sur A, et regarde-le disparaître.

Ce qui est nouveau ici : le 5e argument dans une variable. Comme `regard` pour le miroir, `cote` vaut `0` (devant, tel quel) ou `DERRIERE`. On le passe à `sprite()` à chaque image : `sprite(0, x, y, HEROS, cote)`.

Un seul appui, un seul changement. Si on basculait à chaque image où A est enfoncé, le héros clignoterait 60 fois par seconde tant qu'on tient le bouton. On retient donc dans `avant` si A était **déjà** enfoncé à l'image d'avant. On ne bascule que quand A **vient** d'être enfoncé : `a == 1` (enfoncé maintenant) et `avant == 0` (pas juste avant).

Le texte suit : en haut de l'écran, « DEVANT » ou « DERRIERE ». Les deux espaces après « DEVANT » effacent les dernières lettres de « DERRIERE », plus long de deux lettres.

Essaie partout : sur le mur, derrière ; sur le vide, toujours visible, même en mode DERRIERE. C'est la règle des leçons d'avant, que tu vérifies toi-même.

À toi : fais basculer aussi `MIROIR_X` avec le bouton B. Pour les deux à la fois : `sprite(0, x, y, HEROS, cote | regard)`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **799 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	71 o	← la console, à l'allumage	— la mise en route de la console
main()	253 o	← démarrage	— ta fonction
VBlank	8 o	← la console, à chaque image	— compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	— attend l'image suivante
AttendreVBlank	16 o	← démarrage	— attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte	— attend de pouvoir écrire une case
AdresseCase	10 o	← poser()	— trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte()	— écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage	— range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage	— installe la copie des lutins en mémoire rapide
CopierTuiles	103 o	← démarrage	— copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	— vide les deux cartes de l'écran
LireManette	37 o	← bouton()	— lit les huit boutons
Tuiles	176 o	← CopierTuiles	— 9 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert	— la petite routine qui copie les lutins
"A: DEVANT"	9 o	← texte()	— un texte gravé
"DERRIERE"	8 o	← texte()	— un texte gravé
"DEVANT "	8 o	← texte()	— un texte gravé

Lettres gravées : ADEINRTV:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```
texte() × 3    33 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 5    32 o sur place, dans main() + LireManette (37 o)
sprite() × 1    32 o sur place, dans main()
poser() × 1     13 o sur place, dans main() + AdresseCase (10 o)
image() × 1     6 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : les flèches déplacent un héros ; le bouton A le fait passer
// DEVANT ou DERRIÈRE le mur. En haut, le texte dit où il est.
//
// Ce qui est nouveau : le 5e argument de sprite() dans une variable, cote.
// cote = 0          -> sprite(0, x, y, HEROS, 0)          : devant
// cote = DERRIERE   -> sprite(0, x, y, HEROS, DERRIERE)   : derrière
// Et « un appui = un changement » : on retient si A était déjà enfoncé.

#include <Tuile>      // un dessin de 8 × 8 pixels
#include <poser>      // pose une tuile sur une case du fond
#include <texte>      // écrit un texte à l'écran
#include <bouton>     // lit un bouton de la manette
#include <sprite>     // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — BRIQUE

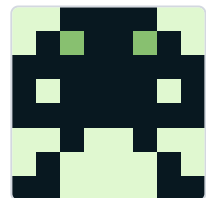
```
Tuile BRIQUE = {                // Le mur : joints sombres (3), briques claires (1).
    "33333333",
    "11131111",
    "11131111",
    "33333333",
    "13111113",
    "13111113",
    "33333333",
    "11131111",
};
```



à l'écran

3. UN DESSIN — HEROS

```
Tuile HEROS = {                // Le héros : les points (.) sont transparents.
    "..####..",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##...##",
};
```



à l'écran

4. UNE VARIABLE — X

```
uint8_t x = 76;                // La place du héros, en pixels : il commence SUR le mur.
```

5. UNE VARIABLE — Y

```
uint8_t y = 72;
```

6. UNE VARIABLE — COTE

```
uint8_t cote = 0;              // 0 : devant ; DERRIERE : derrière le décor.
```

7. UNE VARIABLE — AVANT

```
uint8_t avant = 0;      // A était-il enfoncé à l'image d'avant ? 1 oui, 0 non.
```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    // Le mur : colonnes 8 à 11, lignes 6 à 11 (les pixels 64 à 95 sur 48 à 95).
    for (uint8_t l = 6; l < 12; l++) {
        for (uint8_t c = 8; c < 12; c++) {
            poser(c, l, BRIQUE);
        }
    }
    texte(1, 1, "A: DEVANT");                // Le mode, en haut. « DEVANT » est en colonne 4.

    while (true) {                            // La boucle du jeu :
        image();

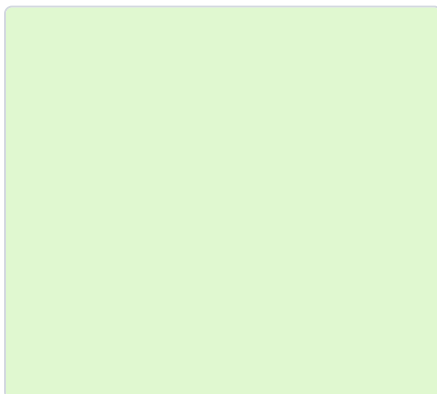
        // Les flèches : un pixel par image, sans sortir de l'écran.
        if (bouton(DROITE) && x < 152) x++;
        if (bouton(GAUCHE) && x > 0) x--;
        if (bouton(BAS) && y < 136) y++;
        if (bouton(HAUT) && y > 16) y--;

        // Le bouton A : on ne bascule que s'il VIENT d'être enfoncé.
        uint8_t a = bouton(A);                // 1 si A est enfoncé maintenant, 0 sinon.
        if (a == 1 && avant == 0) {           // enfoncé maintenant, mais pas juste avant :
            if (cote == 0) {                  // il était devant ?
                cote = DERRIERE;              // il passe derrière,
                texte(4, 1, "DERRIERE");
            } else {                          // il était derrière ?
                cote = 0;                     // il repasse devant.
                texte(4, 1, "DEVANT ");      // (deux espaces : effacer « RE »)
            }
        }
        avant = a;                            // On s'en souvient pour l'image suivante.

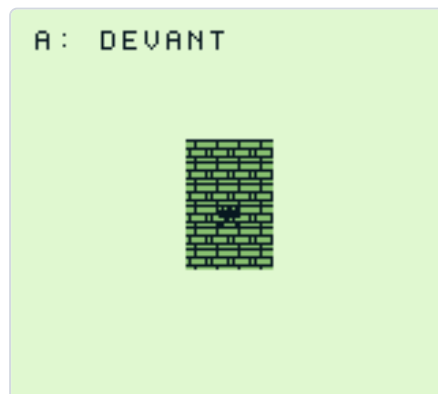
        sprite(0, x, y, HEROS, cote);        // Le 5e argument : 0 ou DERRIERE.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



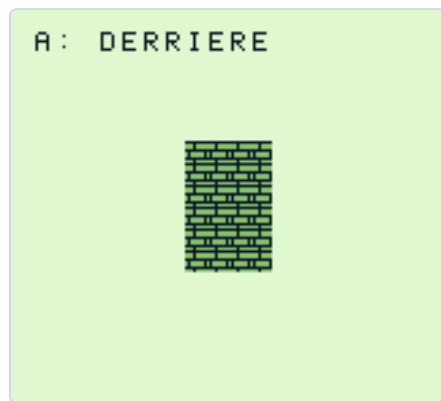
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

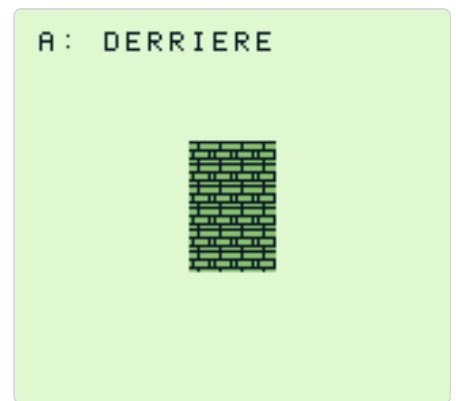
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé



DROITE — l'écran n'a pas bougé



GAUCHE — l'écran n'a pas bougé



HAUT — l'écran n'a pas bougé



BAS — l'écran n'a pas bougé

5. Ce qu'on doit voir

Un grand mur de briques, le héros posé dessus. Les flèches le déplacent ; A le fait disparaître derrière le mur, et A encore le ramène devant.

6. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, le héros est devant le mur (3)
- ✓ A : il passe derrière, on voit la brique (1)
- ✓ et le texte dit DERRIERE (DERRIERE)
- ✓ A encore : il repasse devant
- ✓ DROITE le déplace toujours (76 puis 86)

← [51. Devant, derrière : le bateau passe sous le pont](#)

[sommaire](#)

[53. Devant, derrière : cache-toi du garde](#) →